

PASO 0: Configuración — config.py

La clase RAGConfig (dataclass) carga todas las variables de entorno desde un [.env](#) y crea una instancia global.

Variables relevantes para el pipeline:

Variable	Uso
azure_storage_account, azure_storage_key, azure_container_name	Conexión a Blob Storage (donde están tus docs)
doc_intel_url, doc_intel_key	Azure Document Intelligence (OCR/extracción)
cosmos_endpoint, cosmos_key, cosmosdb_database, cosmosdb_container_cvs	Cosmos DB (donde se guardan los chunks)
azure_openai_url / MODELS_CONFIG	Azure OpenAI (para generar embeddings)
azure_search_endpoint, azure_search_key, azure_search_index	Azure Search (índice de búsqueda)
sublotes_flag	Flag de negocio para detectar "lotes/sublotes" en docs

La validación en `__post_init__` asegura que al menos `AZURE_SEARCH_ENDPOINT`, `AZURE_SEARCH_KEY`, `COSMOS_ENDPOINT` y `COSMOS_KEY` estén presentes.

PASO 1: Crear el índice de Azure Search — create_index.py

Se ejecuta **UNA VEZ**, antes de procesar documentos. Función `create_search_index()`:

1.1 Definición de campos del índice (líneas 24–90)

Campo	Tipo	Para qué
id	String (key)	Identificador único del chunk
chunkId	String	Hash SHA-256 del contenido (dedup)
sectionContent	Searchable (es.microsoft)	Contenido del chunk — buscable por keyword
Title	Searchable	Título extraído del markdown
docTitle	Searchable + filterable	Nombre del documento original
QuestionsText	Searchable	Campo para preguntas generadas (vacío por ahora)
docSummary	Searchable	Resumen del doc (vacío por ahora)
Pages	Filtrable	Rango de páginas "3-5"
topLanguage	Filtrable	Idioma detectado (ej: "es")
nChunk	Int32 filtrable	Nº de chunk dentro del doc
isDeleted	Boolean filtrable	Soft-delete (para re-procesamiento)
isCreated	DateTimeOffset	Timestamp de creación
embedding	Collection(Single) dimensión 1536	Vector embedding (ada-002)

1.2 Vector Search (líneas 93–108)

Configura **HNSW** (Hierarchical Navigable Small World):

- **Métrica:** coseno
- `m=4`, `ef_construction=400`, `ef_search=500`
- Perfil "vector-profile" asociado al campo `embedding`

1.3 Semantic Search (líneas 111–130)

Configura la búsqueda semántica de Azure:

- **Title field:** `Title`
- **Content fields:** `sectionContent`, `QuestionsText`, `docSummary`
- **Keywords fields:** `docTitle`

1.4 BM25 para keyword search (línea 139)

`similarity = BM25SimilarityAlgorithm()`

Resultado: búsqueda híbrida triple (vector + semántica + BM25 keyword).

1.5 Indexer — `create_indexer()` (líneas 146–193)

Crea un *data source* apuntando a Cosmos DB y un indexer que sincroniza automáticamente los documentos de Cosmos → Azure Search. Mapea todos los campos 1:1. Está comentado al final del main — se ejecutaría cuando estés listo.

PASO 2: Procesamiento de documentos — `doc_chunking.py`

La función principal es `get_text_split()`. Recibe el nombre del blob y orquesta todo el pipeline para un documento. Veamos sub-paso a sub-paso:

2.1 Descarga del Blob Storage (líneas 62–66)

```
file_download = asyncio.run(get_specific_file_from_blob_container(
    storageaccountname, storageaccountkey, containername, docId, ...
))
```

Usa `azure_storage_service.get_specific_file_from_blob_container()` que:

- Conecta con `AsyncBlobServiceClient`
- Descarga el blob como bytes (`blob_bytes`)
- Tiene 3 reintentos con delays de 2, 5 y 10 segundos

2.2 Extracción con Document Intelligence (líneas 68-72)

```
docintelligence_service.get_content_from_document():
result = asyncio.run(get_content_from_document(
    file_download=file_download, boolformulas=boolformulas
))
```

Conecta con `DocumentIntelligenceClient` de Azure

Usa el modelo `prebuilt-layout` (extrae tablas, párrafos, fórmulas, idiomas)

Formato de salida: `markdown` (`output_content_format="markdown"`)

Si es PDF con fórmulas: activa features `FORMULAS + LANGUAGES`

Tiene 3 reintentos automáticos

Devuelve un `result` con: `.content` (markdown completo), `.paragraphs`, `.pages`, `.tables`, `.languages`

2.3 Etiquetar páginas (líneas 77-152)

Recorre `result.paragraphs` (para PDFs) o `result.pages` (para otros formatos) y genera un diccionario `page_contents[nº_página]` con los párrafos de cada página.

Después, inserta marcadores de página (`PG1`, `PG2`, ...) dentro del `full_content` al final de cada página. Estos marcadores se usan luego para saber a qué páginas pertenece cada chunk.

2.4 Procesamiento de fórmulas (líneas 155-196)

Si `boolformulas=True`:

Extrae fórmulas por página de `result.pages[i].formulas`

Reemplaza marcadores `:formula:` en el contenido con las fórmulas matemáticas reales

Limpia etiquetas de fórmula residuales

2.5 Procesamiento de tablas (líneas 199-285)

Convierte cada tabla a markdown con `processchunks.convert_table_to_markdown(table)`

Detecta tablas que son footers (aparecen como última tabla en >50% de páginas) y las descarta

Combina tablas que continúan entre páginas (mismo nº columnas, páginas consecutivas)

Genera `tables_by_page[nº]` — lista de tablas markdown por página

2.6 Limpieza de ruido (líneas 291–321) — Timestamp "06 removeContentNoise"

Aplica 14 filtros de limpieza en cadena, usando funciones de `processchunks`:

Función	Qué hace
<code>clean_false_checkbox()</code>	Elimina checkboxes con confianza < 70%
<code>detect_and_remove_headers_footers()</code>	Elimina líneas que se repiten ≥7 veces (headers/footers)
<code>eliminar_figuras_repetidas()</code>	Elimina <code><figure></code> duplicadas
<code>detect_and_remove_duplicate_tables()</code>	Elimina tablas duplicadas (>20% de páginas)
+ 10 regex	Limpia: checkboxes masivos, fórmulas residuales, <code><figure></code> vacías, comentarios HTML, PageHeader, PageFooter, FigureContent, marcadores MIDDLE, etc.

2.7 Chunking (línea 323) — Timestamp "07 ChunkingProcess"

`chunks = processchunks.markdown_percentage(filtered_full_content, 2000, 0.1, 1000, 3000)`

Parámetros: `chunk_size=2000`, `overlap=10%`, `min_tokens=1000`, `max_tokens=3000`.

Detalle de `markdown_percentage()` en `processchunks.py:424-468`:

- Divide por encabezados markdown (H1–H5) usando `MarkdownHeaderTextSplitter` de `LangChain`
- Combina fragmentos pequeños (`<min_tokens=1000` tokens) con un buffer acumulador
- Combina tablas cercanas con `combinar_splits_tablas()` — no separa tablas que estén a <50 chars de distancia
- Subdivide fragmentos grandes (>4000 tokens) con `dividir_si_excede_max_tokens()`:
 - Si tiene tabla → `dividir_chunk_con_tablas()` (no rompe tablas)
 - Si no tiene tabla → `dividir_con_solapamiento()` (10% overlap)

2.8 Enriquecimiento de cada chunk (líneas 340–455)

Para cada chunk se:

- Extrae páginas con regex `PG(\d+)` → genera `page_range` (ej: "3-5")
- Detecta headers (H1–H5) con `detect_title()`, `detect_subtitle()`, etc.
- Hereda headers del chunk anterior: si un chunk no tiene header propio, se le prepende el último header conocido (para dar contexto)
- Asocia tablas y fórmulas por rango de páginas
- Llama a `processchunks.upload_chunks()` pasando todo

2.9 Upload de cada chunk a Cosmos DB — `upload_chunks()` en `processchunks.py:80-148`

Para cada chunk:

1. **Soft-delete previo** (línea 91): Si es el primer chunk del doc (index==1), llama a `mark_existing_chunks_as_deleted()` que busca en Cosmos todos los chunks anteriores del mismo docTitle y les pone `isDeleted=True`.
2. **Genera ID**: UUID sanitizado para Azure Search.
3. **Limpia marcadores** de página del contenido (PG\d+).
4. **Genera chunkId**: hash SHA-256 de docTitle + content + title + page.
5. **Genera embedding** (línea 140):

- `get_embedding(embeddingcontent, Config.azure_openai_emb_name, ...)`

Llama a `openai_service.get_embedding()` que usa `AzureOpenAI.embeddings.create()` con el modelo ada-002 → vector de 1536 dimensiones.

6. **Construye el documento JSON** con todos los campos que coinciden con el schema del índice de Azure Search.
7. **Sube a Cosmos DB** con `upload_doc_cosmos()` → hace `container.upsert_item(data)`.

PASO 3: Azure Search Indexer (sincronización)

Una vez los chunks están en Cosmos DB, el Indexer (creado en `create_indexer()` o manualmente en el portal de Azure) se encarga de:

- Leer periódicamente los documentos de Cosmos DB
- Mapear los campos al índice de Azure Search
- Construir/actualizar el índice de búsqueda híbrida

¿QUÉ FALTA?

1. Fichero orquestador (runner) — NO EXISTE

No hay ningún script que diga "recorre todos los blobs del container y llama a `get_text_split()` para cada uno". Se necesita un fichero tipo `run_ingestion.py` que:

- Liste los blobs del container (la función `list_json_configs_from_blob` ya existe pero para JSONs; se necesita una versión genérica)
- Para cada blob, llame a `get_text_split(blob_name, docId, ...)`
- Gestione paralelismo (`max_workers_docs=3` ya está en config)

Los ficheros `run_cvs.py`, `run_eu.py`, `run_wikipedia.py` existen pero se ignoran.

2. Config.cosmosdb_container — NO EXISTE en el config

En `doc_chunking.py:21` se usa `Config.cosmosdb_container` pero en `config.py` solo existe `cosmosdb_container_cvs`. Esto fallará en runtime. O se necesita añadir `cosmosdb_container` al config, o usar `cosmosdb_container_cvs`.

3. Campos QuestionsText y docSummary siempre vacíos

El índice de Azure Search los define como *searchable* y semánticos, pero `upload_chunks()` siempre los envía como "". Si se quiere búsqueda semántica sobre preguntas generadas o resúmenes, se necesitaría un paso adicional con GPT que genere esos campos.

4. El indexer create_indexer() está comentado

En `create_index.py:200` la llamada está comentada. Deberá descomentarse o crearse el indexer manualmente desde el portal de Azure.

5. datapoints_list.json probablemente no existe

`load_datapoints()` busca un fichero `datapoints_list.json` en el directorio de trabajo. Si no existe, devuelve None y los datapoints se dejan vacíos (no es un error, simplemente no se extraen keywords).

Diagrama del flujo completo

PASO 0: config.py
RAGConfig() carga .env con todas las credenciales y configs

PASO 1: create_index.py → create_search_index()
Crea índice Azure Search: campos + HNSW + Semantic + BM25
(+ create_indexer() para conectar Cosmos → Search)

PASO 2: doc_chunking.py → get_text_split(blob_name, docId)

- 2.1 Blob Storage —download→ bytes del documento
 - └─ azure_storage_service.get_specific_file_from_blob_container()
- 2.2 Document Intelligence —analyze→ result (markdown+meta)
 - └─ docintelligence_service.get_content_from_document()
- 2.3 Etiquetar páginas → PG1, PG2... en el markdown
- 2.4 Procesar fórmulas (si PDF con fórmulas)
- 2.5 Procesar tablas → markdown + combina continuas + elimina footers
 - └─ processchunks.convert_table_to_markdown()
- 2.6 Limpieza de ruido (14 filtros)
 - └─ processchunks.clean_false_checkbox()
 - └─ processchunks.detect_and_remove_headers_footers()
 - └─ processchunks.eliminar_figuras_repetidas()
 - └─ processchunks.detect_and_remove_duplicate_tables()
 - └─ + 10 regex de limpieza en doc_chunking.py
- 2.7 Chunking semántico por headers markdown
 - └─ processchunks.markdown_percentage()
 - └─ MarkdownHeaderTextSplitter (H1-H5)
 - └─ Combinar chunks < 1000 tokens
 - └─ combinar_splits_tablas() (no separar tablas)
 - └─ dividir_si_excede_max_tokens() (max 4000)
 - └─ dividir_chunk_con_tablas() (si hay tablas)
 - └─ dividir_con_solapamiento() (10% overlap)
- 2.8 Por cada chunk:
 - └─ Extraer rango de páginas
 - └─ Detectar headers (H1-H5) y heredar del anterior
 - └─ Asociar tablas/fórmulas por página
 - └─ processchunks.upload_chunks()
 - └─ mark_existing_chunks_as_deleted() (soft-delete)
 - └─ generate_chunkid() (SHA-256)
 - └─ get_embedding() → Azure OpenAI ada-002 → [1536]
 - └─ upload_doc_cosmos() → Cosmos DB upsert

PASO 3: Azure Search Indexer (automático)
Cosmos DB —sync→ Azure Search Index
Búsqueda híbrida: Vector (HNSW) + Semantic + BM25 Keyword